Beat the Bot: Participant Guide (Logs, Rules, Stages)

1. What you build

You build a **decision layer**: a bot that reads each game's state from the game site, asks **Laya** (the open-weight decision model) structured questions, turns Laya's answer into a move, sends the move, and writes a log line for every move. You do not build the games. The games run on the TopGameAI Arena site, and you never run them yourself.

2. The game sites

Stage	Game	Site
1	Snake	https://topgameai.com/arena/snake
2	2048	https://topgameai.com/arena/2048

For how to start a game, send moves, read a game's state, and the exact format of each response, refer to these sites and their documentation. Do not guess formats. Read the responses your games actually return and build your bot on those.

Use your **team name** as the agent name whenever a game asks for one. Leaderboards are public, so never use a personal name.

3. The log (what you submit in your Repo)

Write one JSON object per line in a file ending in .jsonl. Write one line for **each applied move**. Also write a line for every rejected move, marked as rejected, so that nothing is hidden.

3.1 Required fields

Field	Type	Meaning
session_id	string	The game session this line belongs to, as the game gives it to you
game	string	snake or 2048
step	integer	Move number, starting at 1, with no gaps
state_before	object	The game state before this move, exactly as the game returned it
action	string	The move you sent
state_after	object	The game state after this move, exactly as the game returned it
result	string	applied or rejected
decision	object	See 3.2. Required for every move
timing	object	decide_ms, api_ms, turn_ms, integers in milliseconds
ts	string	UTC time in ISO 8601, e.g. 2026-11-07T09:30:12Z
prev_hash	string	The hash of the previous line, or GENESIS-<session_id> on the first line

hash	string	The hash described in 3.3

3.2 The decision block

```JSON
"decision": {
  "questions": ["<the question text you asked Laya>"],
  "probabilities": {"up": 0.61, "down": 0.04, "left": 0.22,
"right": 0.13},
  "chosen": "up",
  "confidence": 0.61,
  "source": "laya"
}
```

- probabilities must have exactly the four keys up, down, left, right. Values are numbers from 0 to 1 and must sum to between 0.98 and 1.02.
- chosen must equal action.
- confidence must equal probabilities[chosen].
- source is laya for a move Laya decided. It may be fallback only for a flagged safety move, described in section 5.3.
- The fallback moves in a session must be at most 10% of its moves.

3.3 Hash chain

Each line's hash is the SHA-256 of the text prev_hash + | + the canonical JSON of the line **without** its hash field. Canonical JSON means keys sorted alphabetically, no spaces, UTF-8, and numbers written without NaN or Infinity.

3.4 Timing

- decide_ms: time from receiving the state to Laya returning its answer.
- api_ms: time from sending the move to the game site until its response arrives.
- turn_ms: time from the previous move's response to this move's response.

Measure with a monotonic clock. Timing from your machine is shown to judges as information, not as proof.

3.5 Keep the final result

When a game ends, save the game's final response in your log folder right away. Game sessions may not stay available for long, so do not rely on the site keeping them.

4. Rules of the games

The official rules are on each game's page: https://topgameai.com/arena/snake and https://topgameai.com/arena/2048. If anything here differs from those pages, follow the game pages and tell the organizers.

4.1 Snake

- You win by eating 15 foods.
- You cannot reverse direction in a single move.
- Hitting a wall or your own body ends the game.
- Step cap: 300 accepted moves. A rejected move does not count as a step.
- Score: foods eaten x 60, plus 100 if you win, capped at 1000.
- Food positions are random, so your log shows the moves but not where food appeared.

4.2 2048

- Merge equal tiles to reach larger ones. Each move slides every tile as far as it can go in the chosen direction, and equal neighbours merge once per move.
- After every move that changes the board, one new tile appears: a 2 most of the time, a 4 occasionally.
- A move that does not change the board is rejected and does not count as a step.
- Step cap: 2000 accepted moves.
- Points: min(1000, max(0, log2(best tile) - 6) x 143 + floor(score / 1000)). The best tile is the largest tile reached, and the score is the in-game score.
- Tiles appear at random, so one run can differ from another with the same bot. Judging uses several runs (section 6).

4.3 Laya rules

- Every move must come from a logged Laya decision.
- You may use a simple rule-based fallback only as a safety measure, for example to avoid a move that is clearly illegal. It must be logged as source: fallback, and it may not exceed 10% of a session's moves.
- No other AI model, search algorithm, reinforcement learning agent, or lookahead planner may choose moves.
- No browser extension, macro, automation tool, or program that plays the game for you outside your visible decision layer.
- During judging, your decision viewer must be visible while your bot runs.

5. Stages and how to advance

Stage	Game	Unlocked by
1	Snake	Nothing. Start here.
2	2048	One verified Snake run with points of at least 600 (ten foods or more)
3 (optional)	Full Doom-style game	One verified 2048 run that reaches a 512 tile or higher

5.1 What makes a run verified

A run is verified when all of these hold:

1. The session is finished, and the organizers can read it.
2. The session's final game, score and step count match the last line of your log.
3. Your log is complete (one line per applied move, with no gaps in step), and its hash chain is intact.
4. Every decision in the log follows section 3.2.
5. Your fallback share is at most 10%.
6. The session was started after your unlock confirmation for that stage, if the stage needs one.

5.2 How runs are scored

- **Snake:** your best verified run.
- **2048:** the median points of your three best verified runs.
- Failed or abandoned sessions stay in the record. Do not delete them. Hiding attempts is a fair-play breach.